Lists, in practice

TC1037 · Algorithmic Thinking · Session 12 · max, min and mean over a column

---

Dr. Leopoldo Rodríguez Salazar

Friday 6 November 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

First, an apology — and a standing invitation

I cancelled tutoring sessions without telling you.

Urgent work came up, I dealt with it, and I lost track of the clock. That is on me, not on you, and it is not how I want this to run.

If you were one of the people left waiting: I am sorry.

Next week I am open.

Book a slot — the link is in the Canvas announcement.

Bring something specific: your midterm paper, the code that will not run, your dataset. Fifteen minutes on a real problem beats an hour of nodding.

Due dates — tonight, Sunday, Tuesday

When	What	What counts as done
Tonight 23:59	**Deliverable 1** `entrega1.xlsx`	An Excel table from a real source. **30–50 rows**, up to **10 columns: 5 text 5 numeric**. In pairs. One upload pe team.
Sunday 23:59	**Homework 6 + Lab 6**	Lists. Both on Canvas now.
Tuesday, in class	**Quick exam 5**	Lists. Same format as the other four.

As of this morning: zero of you have uploaded Deliverable 1. Each missed or late stage costs **5 points** off the project — and the project is 15 % of the course.

Deliverable 1 — the whole thing, on one slide

Where to get data

kaggle.com/datasets · datos.gob.mx · datacatalog.worldbank.org

Or a real company that lets you use its numbers — **a family business counts**, and is often the better project.

The shape

30–50 rows. Up to 10 columns: about five text, about five numeric. You need both kinds — the text columns are what you will group by, the numeric ones are what you will average and plot.

How to not waste the weekend

1. Pick a subject you actually want an answer about.
2. Download the table. Open it. *Look* at it.
3. Delete the columns you will never ask about. Ten is a ceiling, not a target.
4. Save it as `entrega1.xlsx` and upload it as a pair.

Next Friday you write the questions you will ask this table. Pick data that has questions in it.

Where we stopped on Tuesday

```
speeds = [100, 90, 140, 80]
speeds.append(60)
len(speeds)          # 5
speeds[0]            # 100
speeds[len(speeds)-1]    # 60

for x in speeds:          # by value
    print(x)

for i in range(len(speeds)):
    print(i, speeds[i])     # by
    position
```

One name, many values, **in order**. Counting starts at **0**, so the last index is `len(xs) - 1` — and `speeds[5]` is an `IndexError`, not an empty box.

Two ways to walk it. `for x in xs` when you only need the value. `for i in range(len(xs))` when you need *where* it is — the position, or two lists side by side.

Today you will use both, often in the same line.

Today: the three things you will do to every column of your dataset.

Three questions, one shape

```
def mean(xs):
    total = 0
    for x in xs:
        total = total + x
    return total / len(xs)

def maximum(xs):
    best = xs[0]
    for x in xs:
        if x > best:
            best = x
    return best
```

Same four beats every time:

1. start an accumulator
2. walk the list
3. update it
4. `return` it — outside the loop

Minimum is `maximum` with `<`. Write it in thirty seconds and you have three of the five statistics your project needs.

Two traps, both of them yours already

The return inside the loop

```
for x in xs:
    total = total + x
    return total    # <- stops
```

It returns after the *first* value. One indentation level decides whether you get the answer or the first item.

best = 0

```
best = 0
for x in temperatures:
    if x > best: best = x
```

Fine until a column is all negative — then the maximum of `[-3, -8, -1]` comes back `0`, a number that is not in your data. Start from `xs[0]`, not from zero.

Out loud, before I scroll: what does `maximum([-3, -8, -1])` return with `best = 0`?

Both of these cost marks on the midterm. The second is the `n == 0` case wearing a different hat: *what does this do on the awkward input?*

Python already has these — and you are still writing the loop

```
sum(xs)          # the total
max(xs)          # the largest
min(xs)          # the smallest
len(xs)          # how many

sorted(xs)       # a NEW list
xs.sort()        # changes xs
```

Yes, they exist. From week 9 you will use them, and pandas will give you better ones.

This week you write the loop, because in three weeks a library will hand you a column and you need to know what it is doing to it. The loop is the thing being assessed — not the answer.

And now you know why `sum = 0` as a variable name was marked: you just destroyed the function, for the rest of the program.

`sorted()` hands back a new list; `.sort()` changes the one you were given and returns `None`. Same lesson as `print` versus `return`.

sorted() — look at what it actually hands you

```
ventas = [340, 90, 560]

orden = sorted(ventas)
print(orden)     # [90, 340, 560]
print(ventas)    # [340, 90, 560]

ventas.sort()
print(ventas)    # [90, 340, 560]
print(ventas.sort())    # None
```

Smallest first, largest last. `sorted(xs, reverse=True)` turns it round.

Look at lines 4 and 5. `sorted()` hands you a **new** list and leaves yours exactly as it was.

`.sort()` rearranges *your* list and hands back `None` — print it and you get `None`, not the numbers.

Same lesson as `print` versus `return`, one more time: does this hand something back, or does it change what I gave it?

Write it yourself — a sort made out of `maximum`

```
def sorted_desc(xs):
    rest = list(xs)
    out = []
    while len(rest) > 0:
        biggest = maximum(rest)
        out.append(biggest)
        rest.remove(biggest)
    return out
```

Nothing new here but `.remove()`.

Take the largest, put it at the end of a new list, take it out of the old one, repeat until nothing is left. That is a real sorting algorithm, in four lines, built out of the function you wrote ten minutes ago.

How many times? The `while` runs `len(xs)` times, and each `maximum` walks whatever is left: $n + (n-1) + \cdots + 1$.

Ten numbers: `55` comparisons. A thousand: `500,500`.

That is why `sorted()` exists — and why you will call it, not write it, from week 9 on.

A row is a list — and it need not be all numbers

```
fila = ["Norte", "Laptop", 340,
    12]

fila[0]       # "Norte"
fila[2]       # 340
len(fila)     # 4

"Laptop" in fila      # True
"Tablet"  in fila     # False
```

A list may hold text and numbers at once. That is not a curiosity — it is **one line of your** `entrega1.xlsx`: two text columns and two numeric ones.

Out loud: what does `sum(fila)` do?

`TypeError` — you cannot add `"Norte"` to a number.

A **row** mixes words and numbers, so adding a row up means nothing. A **column** is all the same kind of number — that is the thing you total, average and plot. So: pick the column out of the table first, *then* do the arithmetic.

A table is a list of rows

```
tabla = [
  ["Norte", "Laptop", 340, 12],
  ["Sur",   "Mouse",   90,  4],
  ["Norte", "Tablet", 560,  8],
]

tabla[0]         # the first row
tabla[0][2]      # 340
len(tabla)       # 3 rows

ventas = []
for fila in tabla:
    ventas.append(fila[2])
# [340, 90, 560]
```

Two indices: `tabla[fila][columna]`. The first picks the row, the second picks the cell.

Those four lines **pull a column out of a table** and hand you the flat list you already know how to work with:

`mean(ventas)` → `330.0`
`maximum(ventas)` → `560`

This is your spreadsheet. Nothing else about it is new.

The question you will actually ask: “which branch sells most?”

```
ventas_norte = []
for fila in tabla:
    if fila[0] == "Norte":
        ventas_norte.append(fila[2])

print("Norte:", sum(ventas_norte))
```

An `if` inside a `for`, over rows instead of numbers.

This is deliverable 2. “Which branch sells most? Which product? Who are the best clients?” — every one of those is this shape: filter the rows by a text column, collect a numeric column, run a statistic.

Out loud: what does this print for the table above?

`Norte: 900` — 340 and 560, not 90.

If you want it: every cell, not just one column

```
for fila in tabla:
    for celda in fila:
        print(celda)
```

A loop inside a loop: the outer one walks the rows, the inner one walks the cells of *that* row. It runs `rows` × `columns` times — 12 for our table.

You do not need it this week. You will in week 8, and when you do, the *how many times* sentence is the whole point.

Why this is the project, not an exercise

```
# the column from tabla
ventas = [340, 90, 560]

def descriptiva(xs):
    return (total(xs),
            mean(xs),
            maximum(xs),
            minimum(xs),
            len(xs))
```

The final program owes **five descriptive statistics**. They are the functions from this hour, run on the column you pulled out of `tabla`.

Homework 6 is exactly this, with the empty and all-negative lists handled. **Write the loops** — no `sum()`, `max()` or `min()` inside your four functions.

In week 9 `pandas` hands you the column instead. Nothing else changes.

So tonight's upload matters: no table, no column, nothing to run this on.

A list is a column.

A loop is a question you ask it.

Tonight 23:59 — `entrega1.xlsx`, in pairs

Sunday 23:59 — Homework 6 and Lab 6

Tuesday — Quick exam 5, in class

And book a tutoring slot. I am free next week and the link is in Canvas.